Shared Files

1 Basic Idea

Shared File

A **shared file** is a file that appears in more than one directory, so different users or directories can access the same underlying file.

- Shared files are useful when multiple users work on the same project.
- A shared file may appear in directories belonging to different users.
- The connection from another directory to the shared file is called a **link**.
- With links, the file-system structure is no longer a pure tree.
- It becomes a **directed acyclic graph**, or **DAG**.

2 Shared File as a DAG

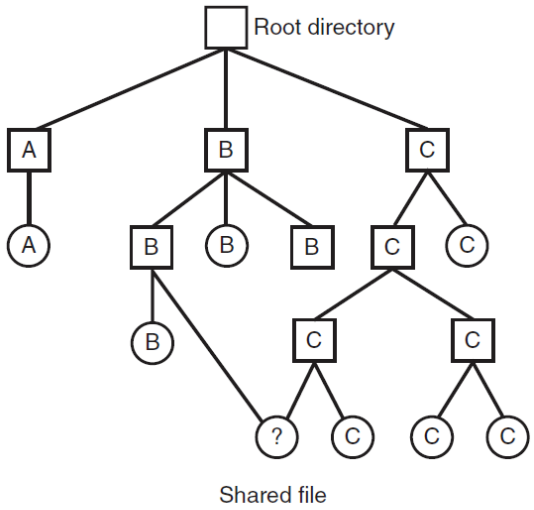

Figure 4-16. File system containing a shared file.

Figure 1: A file system containing a shared file.

- In a tree, each file or directory has exactly one parent path.
- With sharing, the same file can be reached through more than one directory.
- The shared file has multiple incoming links.
- This makes maintenance more complicated because the system must handle multiple names for the same file.

3 Why Direct Disk Addresses Cause Trouble

Problem with Copying Addresses

If directory entries contain direct disk addresses, linking a file by copying those addresses can break sharing.

- Suppose C owns a file and B links to it.
- If disk addresses are copied into B's directory, B and C now have separate address lists.
- If B appends data, only B's directory entry may be updated.
- If C appends data, only C's directory entry may be updated.
- The two users may stop seeing the same file contents, defeating the purpose of sharing.

4 Solution 1: Hard Links

Hard Link

A **hard link** makes multiple directory entries point to the same internal file data structure, such as an i-node.

- Disk block addresses are stored in the file's data structure, not directly in each directory entry.
- In UNIX-like systems, this data structure is the i-node.
- Each directory entry points to the same i-node.
- Since the i-node is shared, updates to the file are visible through all hard links.
- The i-node stores a **link count**.
- The link count records how many directory entries point to the file.

5 Hard-Link Count Example

- Before linking, only C's directory points to the file, so the count is 1.
- After B creates a hard link, both B's and C's directories point to the same i-node, so the count becomes 2.
- If C removes the file name from C's directory, the i-node is not deleted.
- The count decreases to 1 because B's directory still points to the file.
- The file is deleted only when the link count becomes 0.

6 Hard-Link Drawback

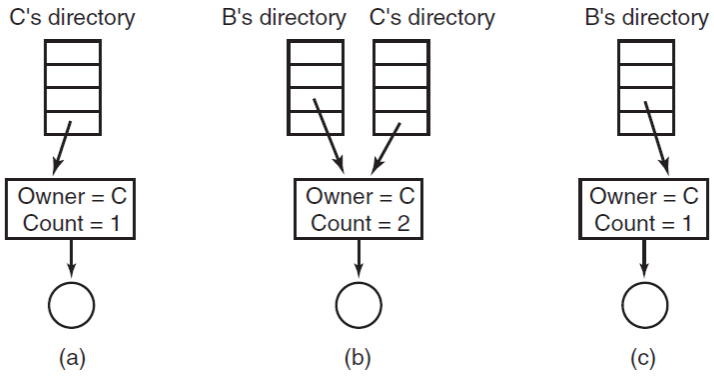

Figure 4-17. (a) Situation prior to linking. (b) After the link is created. (c) After the original owner removes the file.

Figure 2: Link count before linking, after linking, and after the original owner removes the file.

Ownership and Accounting Problem

With hard links, the file may continue to be charged to the original owner even after that owner's directory entry has been removed.

- Creating a hard link does not change the owner recorded in the i-node.
- If C owns the file and B links to it, the owner remains C.
- If C deletes C's name for the file, B may still keep the file alive.
- The system cannot easily find and remove every directory entry that points to the i-node.
- This can affect accounting and disk quotas.

7 Solution 2: Symbolic Links

Symbolic Link

A **symbolic link** is a small special file that stores the path name of another file.

- B can link to C's file by creating a new file of type **LINK**.
- The link file stores the path name of the target file.
- When the link is opened, the operating system reads the path and follows it to the real file.
- The symbolic link does not point directly to the target file's i-node.
- Removing the symbolic link does not affect the target file.

8 Symbolic-Link Behavior

- If the original file is deleted, the symbolic link remains.

- Later attempts to open the symbolic link fail because the target path no longer exists.
- This is called a dangling or broken symbolic link.
- Symbolic links can cross disk boundaries.
- They can also refer to files on remote machines if the path includes network location information.

9 Hard Links vs Symbolic Links

Point	Hard Link	Symbolic Link
What it stores	Direct reference to the same i-node	Path name of the target file
Target deletion	File remains until link count is 0	Link becomes broken if target is deleted
Effect of removing link	Decreases link count	Removes only the link file
Disk boundaries	Usually limited within one file system	Can cross disks and networks
Overhead	Low lookup overhead	Extra lookup and path parsing
Extra storage	No separate path file needed	Needs a link object and stored path

10 General Problems with Links

- A linked file can have more than one path name.
- Programs that recursively scan directories may find the same file multiple times.
- Backup programs may accidentally store multiple copies of one shared file.
- Restoring such backups on another system may duplicate the file instead of preserving the link.
- File-system tools must be aware of links to avoid incorrect accounting, copying, or traversal.

Final Point

Shared files make collaboration easier by allowing multiple directory entries to refer to the same file, but links complicate deletion, ownership, accounting, traversal, and backup behavior.